CPSC 483 MIDTERM — FRONT (Ch1 Landscape · Ch2 Preprocessing)

Ch1 What is ML?

AI = machines simulating human intelligence; **ML ⊂ AI** (learn from data w/o being explicitly programmed); **Deep Learning ⊂ ML** (multi-layer neural nets).
Samuel 1959: ability to learn w/o being explicitly programmed. **Simon 1983**: any change that lets a system do better the 2nd time on same task/population. **Mitchell 1997**: learns from **Experience E** w.r.t. **Task T** and **Performance P** if P on T improves with E. (Spam: T=flag spam, E=labeled emails, P=accuracy.)
Learning = acquiring knowledge: facts, patterns, concepts (generalize: 0,2,4→2n), rules (P→Q), models (v=d/t), skills. **Traditional**: Data + Program → Output (hand-written rules). **ML**: Data + Output → Program.

Why use ML (4 reasons)

Problems needing lots of fine-tuning / long rule lists (spam) · Complex problems w/ no good traditional solution · **Fluctuating environments** (just retrain on new data → adapts automatically) · **Insights** into complex problems/large data (data mining; inspect the model to understand the problem).
Drivers: data explosion, real-time decisions, shift rule-based → data-driven.

Applications

Pattern recognition (faces, speech, handwriting) · pattern generation (image/motion synthesis) · anomaly detection (fraud, sensors) · prediction (stocks, health/ECG, weather) · autonomous cars (levels 0–5) · scene labeling · recommender systems (collaborative = similar users; content-based = similar items).

Main challenges

ML is **ill-posed**: limited sample of the world. Goal = **generalize** beyond dataset, not fit noise.
Occam's razor (upper bound): simplest explanation tends to be right. **Einstein** (lower bound): as simple as possible, but not simpler.
Students: **A** not interested → class 50%/test 47% = **underfit (high bias)**; **B** memorizes → 98%/69% = **overfit (memorizing, high variance)**; **C** learns concepts → 92%/89% = **best fit**.
Underfit: too simple, misses patterns, poor on train AND test. **Overfit**: too complex, fits noise, great train / poor test.
Others (textbook): insufficient data, nonrepresentative data (sampling bias), poor-quality data, irrelevant features.
Inductive bias = assumptions that define model selection; needed to generalize (else just memorize). **Restriction bias** = limits the set of models considered (e.g., only linear). **Preference bias** = prefers some models over others (e.g., simpler).

Taxonomy (criteria are NOT exclusive)

1. **Supervision**: supervised / unsupervised / semi- / self-supervised / RL
2. **Incremental?** online vs batch
3. **Instance-based** (compare new points to known, e.g., KNN) vs **model-based** (build predictive model, e.g., linear reg.)

Task	Output	Goal / examples
Classification	discrete (labels)	find **decision boundary**; spam, hot/cold. Nominal (dog/cat) vs ordinal (bad<good<excellent)
Regression	continuous number	find **best-fit function**; tomorrow's temp, price
Clustering	groups, NO labels	unsupervised; customer segmentation

Supervised: given labeled {(X,Y)}, learn f: X→Y ("training"). Methods: least squares, GD, KNN, ANN, Naïve Bayes, trees, random forest, SVM, logistic reg. Most work for both class. & regression.
Multi-class: one class per instance, many choices (hot/warm/cold). **Multi-label**: several labels at once (dog AND cat in image).

- **Unsupervised** (no labels; if labels exist, use supervised): **clustering** (partitional, hierarchical, density-based), **association rules** (Apriori, FP-growth), **feature selection / dim. reduction** (correlation matrix, info gain, LDA, PCA), anomaly/outlier detection, visualization.
- **Assoc. rule X→Y**: **Support** s = σ(X∪Y)/|T| ; **Confidence** c = σ(X∪Y)/σ(X). Ex {Bread,Milk}→{Diaper}: s=2/5=0.4, c=2/3=0.67.

Semi-supervised (weak supervision)

- Why: labels expensive (experts, time); unlabeled data cheap/abundant. Uses **small labeled + large unlabeled**. Unlabeled data must be relevant: p(x) must carry info about p(y|x).
- **Inductive** methods (train a model): **self-training / pseudo-labeling** (train on labeled → predict unlabeled → add most-confident pseudo-labels → retrain; a wrapper method), **unsupervised pre-processing** (pre-train e.g. autoencoder; final model trained only on human labels), or put unlabeled into the objective.
- **Cluster-then-label**: cluster; unlabeled points get the majority label of their cluster. **Active learning** (transductive): pick which unlabeled samples are most useful to label by hand.

Self-supervised

- Generates its own labels/targets **from the data itself** via **pretext tasks** (mask a word "The cat [ ] on the mat", image completion, rotation prediction, jigsaw, colorization, autoencoders) → learned representations reused on a downstream task (= **transfer learning**).
- vs Supervised: both optimize vs a known target, but supervised needs human labels. vs Unsupervised: both use unlabeled data, but unsup. finds structure; self-sup. creates a prediction task. vs Semi: semi needs a small labeled set; self-sup. needs none.

Reinforcement Learning

- **Agent** acts in **environment** → gets **reward**/penalty → learns a **policy**; goal = max long-term reward. **Explore** (try actions) vs **exploit** (favor best-reward actions). Training data optional.
- Ex: AlphaGo/AlphaZero, robot walking, self-driving. **Model-based**: dynamic programming / Bellman (policy & value iteration). **Model-free**: Monte-Carlo, temporal-difference, SARSA, Q-learning, DQN.
- RL is **both** supervised (policy improved by reward) and unsupervised (no explicit goals; trial & error) → **neither/both**.

Other categories

- **Deep learning** = technique (many-layer NN), NOT a paradigm; auto feature learning, needs lots of data + GPUs. Traditional ML: manual features, small data, cheaper, more interpretable.
- **Batch/offline**: train on all data at once; must retrain from scratch to update; poor with concept drift.
  Online/incremental: learn as data streams in; adapts to drift, less memory; good for fraud, stocks, big data.
- **Transfer learning**: reuse model trained on source task (VGG, ResNet, BERT, GPT). Concerns *where the model starts*; online concerns *how it updates* → not exclusive.
- **Ensemble**: combine many models (random forest = bootstrapped trees). Costs compute.
- **Choosing**: labels? → sup/unsup/semi; goal → reg/class/cluster/assoc; optimal actions → RL; compute → DL; big/streaming data → online. **No Free Lunch**: no model is best for every problem.

Ch2 Data Preprocessing

ML steps: big picture (objective → which metric; current baseline) → get data → EDA/visualize → prepare data → select & train → fine-tune → present → launch/monitor.

- **Instance** = one row/sample. **Feature** = input column (attribute); **feature vector** = all features of one instance. **Label/class** = target output column.
- **Training set** teaches the model; **test set** evaluates it. Test data is **NEVER** used in training.
- **EDA**: first step, data-driven/model-free, interactive & visual; summary stats (df.describe(), info()). **Univariate** (1 var: histogram), **bivariate** (2: scatter, corr), **multivariate** (3+: pairplot, heatmap).

Train-test split

- Typically **80% train / 20% test**. Train as big as possible; test big enough to be representative.
- Do EDA **only on train** → avoids **data snooping bias** (peeking at test leads to optimistic model choice).
- Don't take first 80% (data may be sorted → test unlike train).
- **Random**: every row equal chance; may NOT keep proportions. `train_test_split(df, test_size=0.2, random_state=42)`
- **Stratified**: split into homogeneous strata (e.g., income cat, class) → test has same proportions as full data. add `stratify=df["col"]` . Use for imbalanced/important categories.

Cross-validation

- **k-fold**: split train into k folds; train on k−1, validate on 1; repeat k times; average score. `cross_val_score(model, X, y, cv=5)` . Less dependent on one lucky split; costs k× training.
- **Train / validation / test**: validation picks model & hyperparameters; test used **once** at end. Process: choose best via validation → **retrain on all training data** → evaluate once on test = generalization error.

Variables & encoding

- **Numerical**: natural order, can average (continuous/discrete: age, weight). **Categorical**: no order to average (nominal: color; ordinal: S/M/L).
- **Num → Cat (binning/discretization)**: **equal-width** = same-size ranges (bin counts differ, e.g. 2/4/3); **equal-frequency** = ~same # items per bin (ranges differ, e.g. 3/3/3); custom = user thresholds.
- **Cat → Num**: **Ordinal/label** encoding = one integer per category [Green,Red,Black,Green]→[1,2,3,1]; implies order/distance (fine for ordinal). **One-hot** = one 0/1 column per value (sparse matrix); no fake order but many columns if many categories.
- Practice "Standing" [Fr,Jr,Fr,Sr]: ordinal Fr0 Jr1 Sr2 → [0,1,0,2]; one-hot (Fr,Jr,Sr) → [1,0,0],[0,1,0],[1,0,0],[0,0,1].

Feature scaling — why?

Algorithms (GD, SVM, KNN) are biased toward large-unit features (income $20k–200k swamps age 18–80). Scale → common range w/o distorting differences. **Fit scaler on train only**, then transform test.

Min-Max (normalization) $x' = (x - \min)/(\max - \min)$ → [0,1]. Ex (37−16)/(100−16)=0.25.
Any range [a,b]: $x' = (x-\min)/(\max-\min)\cdot(b-a)+a$; [−1,1]: 2x'−1. Bounded but **very sensitive to outliers**. `MinMaxScaler(feature_range=(0,1))`
Standardization (z-score) $z = (x-\mu)/\sigma$ → mean 0, std 1; range NOT bounded; less outlier-sensitive. `StandardScaler()`
Custom: log/sqrt for skewed data, ratios (rooms/household). `FunctionTransformer(np.log)`

Practice Age[16,28,30,23,60]: [0,1]→0, .273, .318, .159, 1; [−1,1]→−1, −.455, −.364, −.682, 1. Income[10k,2k,6k,5k,25k]: [0,1]→.348, 0, .174, .130, 1; [−1,1]→−.304, −1, −.652, −.739, 1.

Missing data

- Count NA (`df.isna().sum()`); watch hidden missing values (0, −1, impossible values). Outliers too.
- **Drop** rows/cols (`dropna()`) — loses data; or **impute** mean/median/most-frequent (`fillna()` , `SimpleImputer(strategy="median")`). Median is robust to outliers.

Data processing pipelines

- Chain of transforms run in order: impute → encode → scale → model. `Pipeline([("imp",SimpleImputer()), ("sc",StandardScaler()),("m",model)])`
- `ColumnTransformer` : different steps per column type (num: impute+scale; cat: one-hot).
- Why: reproducible, less code, same steps on train & new data, and **prevents data leakage** — pipeline.fit fits every transformer on train only; test is only transformed. Works inside cross-validation/grid search.
- fit() = learn params (min/max, μ/σ); transform() = apply; fit_transform() on train ONLY.

BACK (Ch3 Evaluation · Ch4 Linear Models · Ch5 SVM)

Ch3 Model Evaluation

Why evaluate? Compare models quantitatively; tune **hyperparameters** (settings of the training algorithm / preprocessing, not learned from data).

Actual \ Pred	Positive	Negative
Positive (P)	**TP**	**FN** (Type II, miss)
Negative (N)	**FP** (Type I, false alarm)	**TN**

True/False = prediction matched truth? Positive/Negative = what model *predicted*. P = TP+FN, N = FP+TN, All = P+N. `tn,fp,fn,tp = confusion_matrix(y,ŷ).ravel()` (sklearn: rows actual, cols predicted, order neg→pos)

Accuracy = (TP+TN)/All **Error** = (FP+FN)/All = 1 − acc
Precision (exactness) = TP/(TP+FP) — of predicted +, how many right?
Recall = Sensitivity = TPR (completeness) = TP/(TP+FN) = TP/P — of actual +, how many found?
Specificity = TNR = TN/(TN+FP) = TN/N **FPR** = FP/N = 1 − specificity
F1 = 2·P·R/(P+R) (harmonic mean; high only if both high; use when FP & FN both matter)

Class imbalance: accuracy dominated by majority class; 5% positives → "always negative" = 95% accurate yet useless. Use precision/recall/F1/ROC-AUC, stratified sampling, class weights.

Precision vs recall trade-off (inverse): classifier score vs **threshold**. ↑threshold → ↑precision ↓recall; ↓threshold → ↑recall ↓precision. Favor **recall** when FN costly (cancer, fraud); **precision** when FP costly (spam filter, kid-safe videos).

PR curve: precision & recall vs threshold (`precision_recall_curve`). Prefer PR over ROC when positives are rare / care about FP.

ROC: TPR (recall) vs FPR across thresholds. Good = hugs **upper-left**; random = **diagonal**. **AUC**: 1 = perfect, 0.5 = random; compares classifiers on average.

Multi-class from binary: **OvA/OvR** = one classifier per class vs all others (N classifiers; pick highest score). **OvO** = one per pair: N(N−1)/2 classifiers (10 classes → 45); each trains on a small subset (good for SVM, which scales poorly).

Multi-label: multiple non-exclusive labels per instance [Dog=T, Cat=T, Bird=F]. Multi-class: exactly one of many.

Cancer practice (TP90 FN210 FP140 TN9560, All 10000): acc 96.50%, err 3.50%, prec 90/230=39.13%, recall 90/300=30.00%, spec 98.56%, F1≈0.340 → high accuracy hides poor recall (imbalance!).

Credit practice (+=High; actual H,L,L,L,H,L; pred H,H,L,H,H,H): TP2 FP3 TN1 FN0 → acc 50%, err 50%, prec 0.40, recall 1.00, F1≈0.571.

Ch4 Linear Regression

Simple: one feature $\hat{y} = \theta_0 + \theta_1 x$. **Multiple**: $\hat{y} = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n = \theta^T x$; matrix form y = Xθ (X has column of 1s). `LinearRegression().fit(X,y)`; intercept_, coef_.

Model steps: pick model family (linear, quadratic…) → fit = find best parameters.

Residual = $y_i - \hat{y}_i$. Don't just sum residuals (± cancel) → square them.

MSE = $(1/N)\Sigma(y_i - \hat{y}_i)^2$ (cost function; penalizes big errors).
RMSE = √MSE (same units as y → interpretable). SSE = $\Sigma(y_i - \hat{y}_i)^2$. (MAE = mean|error|, less outlier-sensitive.)

Normal equation (closed form): $\theta = (X^T X)^{-1} X^T y$. Cost $O(n^{2.4})$–$O(n^3)$ in # features n: 2× features → 5.3–8× time. Linear in # instances. Slow w/ many features → use GD.

Gradient Descent

Iterative: start w/ initial θ (e.g., 0/random) → step opposite gradient to reduce MSE → stop when converged. $\theta \leftarrow \theta - \eta \nabla_\theta \mathrm{MSE}(\theta)$, $\nabla \mathrm{MSE} = (2/m) X^T (X\theta - y)$.

- MSE for linear reg. is **convex** → one global min, GD reaches it. Non-convex → may stop at **local** min / plateau.
- **Learning rate η**: too small → very slow; too large → overshoots/diverges. **Scale features** first (else elongated bowl, slow).

Variant	Per step uses	Notes
Batch	ALL samples	smooth, stable, straight path; slow on big data
Stochastic	1 random sample	fast, zig-zag/noisy, can escape local min, never fully settles (use decaying η); good for big/online data
Mini-batch	small random batches	compromise; reshuffle each epoch; GPU-friendly; less noisy than SGD

Polynomial regression

- Add powers/combos of features, then fit linear model → fits nonlinear data. Still **linear in the parameters θ** → is linear regression.
- `PolynomialFeatures(degree=3)` on a,b adds a^2, a^3, b^2, b^3, ab, a^2b, ab^2 → **combinatorial explosion** of features.
- Degree 1 underfits, 2 fits, 300 wildly overfits (wiggles through points).

Learning curves & bias/variance

- Plot train & validation error vs training-set size/iterations.
- **Underfit**: both curves plateau close together at **high** error; more data won't help → better/complex model, more features, higher degree, less regularization.
- **Overfit**: low train error, **gap** to higher validation error → more data (until val ≈ train), simpler model, regularize.
- Gen. error = **Bias²** (wrong assumptions, e.g., linear for quadratic → underfit) + **Variance** (over-sensitive to small data changes, many DOF → overfit) + **Irreducible** (noise; fix only by cleaning data/sensors/outliers).
- ↑complexity → ↓bias ↑variance; ↓complexity → ↑bias ↓variance. Complex = low bias + high var; simple = high bias + low var. Goal: sweet spot.

Regularization

- Cost = $\ell(w) + \alpha R(w)$, α>0. **α = 0** → plain linear reg.; larger α → stronger penalty, weights → 0, simpler model (↑bias ↓var). Choose α by **cross-validation**. Scale data first. Don't regularize θ_0.
- **Ridge (L2)**: $+ \alpha \Sigma \theta_i^2$; shrinks weights near (not exactly) 0; keeps all features. `Ridge(alpha=0.1)`
- **Lasso (L1)**: $+ \alpha \Sigma |\theta_i|$; drives some weights exactly 0 → **feature selection**, sparse model (diamond corners). `Lasso(alpha=0.1)`
- **Elastic Net**: mix, ratio r: $\mathrm{MSE} + r \cdot \alpha \Sigma|\theta_i| + (1-r) \cdot \alpha \Sigma \theta_i^2$ (r=0 Ridge, r=1 Lasso). Best with many / correlated features. `ElasticNet(alpha, l1_ratio=0.5)`
- **Early stopping**: stop GD when validation error hits its minimum; with SGD/mini-batch wait until it stays above min a while, then roll back to best params.

Logistic regression (classification!)

- Weighted sum t = θᵀx → **sigmoid** $\sigma(t) = 1/(1+e^{-t}) \in (0,1)$ = estimated probability p̂. Predict 1 if p̂ ≥ 0.5 (t ≥ 0), else 0. Binary (multiclass via OvR / softmax).
- Log-loss: $c = -\log(\hat{p})$ if y=1, $-\log(1-\hat{p})$ if y=0. $J(\theta) = -(1/m)\Sigma[y \log \hat{p} + (1-y)\log(1-\hat{p})]$. Convex, **no closed form** → gradient descent.

Ch5 SVM (math detail not required)

- Supervised (Vapnik, 1990s); linear/nonlinear classification & regression. Finds hyperplane that **maximizes the margin** ("maximum margin classifier", widest street).
- **Margin** = width between the support-vector lines. **Support vectors** = points closest to hyperplane; hardest to classify; **only they** define the boundary (adding far points changes nothing).
- Decision: $w^T x + b = 0$; margin lines $w^T x + b = \pm 1$. **Sensitive to feature scaling** → always scale.
- **Hard margin**: all points off the street & correct side; needs linearly separable data; very **outlier-sensitive**.
- **Soft margin**: balance wide street vs margin violations. Slack ξ: **ξ=0** correct side, no violation; **0<ξ<1** correct side but inside margin; **ξ>1** misclassified. Objective $\min \tfrac{1}{2}\|w\|^2 + C\Sigma\xi_i$ s.t. $y_i(w^T x_i + b) \geq 1 - \xi_i$ (hinge loss).
- **C** (pick by CV): large C → few violations, narrow margin, less regularization, **overfit**; C→∞ = hard margin; small C → wider margin, more violations, more regularization (underfit). Overfitting? ↓C.
- **Imbalanced data**: class-weighted slack $C \cdot N/(2N_+)\Sigma\xi^+ + C \cdot N/(2N_-)\Sigma\xi^-$ (N_++N_-) → minority errors cost more so majority doesn't dominate (sklearn class_weight="balanced").

Nonlinear SVM & kernels

- Not linearly separable → map to higher dimension where it is (1-D x → 2-D (x, x^2)). Explicit polynomial features explode in count.
- **Kernel trick**: SVM dual only needs **dot products**; K(x,z) = φ(x)·φ(z) computed in original space **without transforming** → cheap high/infinite-dim features.
- Ex $\varphi(x_1,x_2) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$; x=(2,3), y=(1,4): φ(x)·φ(y) = 4+144+48 = **196** = $(x^T y)^2 = 14^2$. Practice x=(1,2,−1), y=(0,−2,1): $x^T y = -5 \rightarrow 25$; φ·φ = 0+16+1+0+0+8 = 25 ✓.
- **Polynomial** $K = (x \cdot z)^d$ (sklearn: $(\gamma x \cdot z + \text{coef0})^d$); d=1 linear, d=2 quadratic. ↑degree/coef0 → more complex. `SVC(kernel="poly", degree=3, coef0=1, C=5)`
- **Similarity features**: similarity of each instance to **landmarks**; simplest = landmark at every instance → m instances ⇒ m features (huge if large data).
- **Gaussian RBF** $K = \exp(-\gamma\|x - z\|^2)$: ≈1 when close, ≈0 when far. ↑γ → narrower bell, wiggly boundary, **overfit**; ↓γ (large σ) → smoother, like linear. **Large C + large γ → overfit; small C + small γ → underfit.**
- **Which kernel**: linear first (LinearSVC much faster on big data) → RBF if data not too large → others via hyperparameter search (grid search + CV).

SVM Regression (SVR)

- Reverse goal: fit as many points **inside** the street/tube as possible, limit points outside. Tube width = **ε**.
- **ε-insensitive loss** $L = \max(0, |y - f(x)| - \varepsilon)$: 0 inside tube, linear outside. Points inside don't affect model (ε-insensitive); support vectors = points on/outside tube.
- Larger ε → wider tube, fewer SVs, simpler; smaller ε → narrow tube, more SVs. Nonlinear via kernels `SVR(kernel="poly", degree=2, C, epsilon)`; large C → less regularization.

Common T/F traps (give reasoning!)

- *High accuracy ⇒ good model? F — imbalance (95% by always "neg").*
- *Test set may be used to tune hyperparameters? F — use validation/CV; test once.*
- *Polynomial regression is nonlinear regression? F — linear in θ.*
- *Lasso can zero out weights; Ridge can't (only near 0). T.*
- *More training data fixes underfitting? F — fixes overfitting (variance).*
- *Larger C in SVM = more regularization? F — less (narrower margin, overfit risk).*
- *Deep learning is a learning paradigm like RL? F — a technique usable in any paradigm.*
- *Removing non-support-vector points changes the SVM? F.*
- *Min-max scaling is robust to outliers? F — standardization is less sensitive.*
- *GD always finds global min? F — only if convex (e.g., MSE of linear reg.).*
- *OvO trains N(N−1)/2 classifiers; OvA trains N. T.*
- *Logistic regression has a closed-form solution? F — GD.*